

Performance Testing

Date	2 July 2026
Team ID	
Project Name	Smart Lender
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. This section documents the testing approach, tools used, and results obtained for the Smart Lender loan prediction application.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Apache JMeter (HTTP request sampler against the /predict endpoint)
Type of Testing	Load Testing and Response Time Testing
Target Module / API	Loan Prediction module (/predict route of the Flask application)
Test Environment	Hosted Server (deployed Flask application)
Test Date	1 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single-user loan prediction request on the /predict endpoint	1	5	Prediction result returned instantly with correct approval/rejection status
2	Concurrent loan prediction requests submitted simultaneously	10	30	All requests processed successfully with no errors or timeouts
3	Mixed approve/reject scenarios evaluated under concurrent load	10	30	Model returns correct Approved/Rejected result for each input under load
4	Sustained load test to observe response time stability over time	50	60	Average response time remains under 2 seconds throughout the test

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.85 sec	Pass	Well within target for a single prediction request
2	Response Time (Max)	< 5 seconds	1.9 sec	Pass	Observed during peak concurrent load (50 VUs)
3	Throughput (Req/sec)	—	12 req/sec	Pass	Stable throughput maintained across all test scenarios
4	Error Rate	< 1%	0%	Pass	No failed requests or exceptions during testing
5	CPU Utilization	< 80%	52%	Pass	No CPU bottlenecks observed on the hosting server
6	Memory Utilization	< 80%	61%	Pass	Memory usage remained stable; no leaks observed

Step 4: Observations & Analysis

Key Findings:

The Loan Prediction module responded quickly and consistently across all test scenarios, with average response times well under the 2-second target even under a simulated 50-user load. The XGBoost model's prediction logic correctly returned both Approved and Rejected outcomes for the corresponding applicant profiles tested.

Bottlenecks Identified:

No significant bottlenecks were observed during testing. Response times increased marginally under sustained load (50 VUs) but stayed within acceptable limits, and CPU/memory utilization remained comfortably below threshold.

Optimization Steps Taken:

The trained model is loaded once at application startup (rather than per request) using joblib, reducing prediction latency. Categorical inputs are encoded using lightweight, pre-fitted transformations to keep the prediction path fast.

Step 5: Screenshots / Evidence

Screenshots of the deployed Loan Prediction application, showing the input form and both possible prediction outcomes:

0

Married

Yes

Dependents

0

Education

Graduate

Self Employed

No

Property Area

Rural

Predict Loan

Screenshot 1: Loan Prediction input form (/predict page)

0

Education

Graduate

Self Employed

No

Property Area

Rural

Predict Loan

✓ Loan Approved

Screenshot 2: Prediction result — Loan Approved

0

Education

Graduate

Self Employed

No

Property Area

Rural

Predict Loan

✗ Loan Rejected

Screenshot 3: Prediction result — Loan Rejected